

SentinelCut — Team Plan & Ownership (3-Person MVP)

This adapts the MVP Scope Checklist's 4-role breakdown into a 3-person build. Keep this file at the repo root — it's the shared source of truth for who owns what and how we integrate.

1. Roles

The original scope splits work into AI/ML, Backend, Audio/Video, and Frontend. With 3 people, **Backend and Audio/Video are merged** — in practice the audio/video edit step is just a stage inside the backend's Celery pipeline (ASR → detection → audio edit → mux), so splitting them across two people just adds handoff friction.

Person	Owns	Key deliverables
P1 — AI/ML	Whisper STT + language handling + full detection pipeline (Scope §3)	Normalization (§3.1), dictionary matching (§3.2), regex/fuzzy matching (§3.3), outputs valid detection JSON (§3.5)
P2 — Backend + Audio/Video (team lead)	FastAPI, Postgres schema, Celery/Redis queue, auth, FFmpeg/PyDub beep+mute, sync, export	RESTAPI, job queue, DB schema, censored audio/video export, session auth
P3 — Frontend	Upload UI, dashboard, transcript/timeline viewer	Upload flow, recent-files dashboard, accept/remove-detection UI, exports view

Why the team lead takes Backend: it's the integration hub — it consumes P1's output and serves P3's UI — so it's the natural place to sit if you're also unblocking people and reviewing PRs.

2. The Integration Contract (lock this on Day 1)

Every detection produced by the AI/ML pipeline must match this shape. Backend and Frontend can build against this **before** the real pipeline exists — Frontend uses mock data, Backend validates against it.

```
json
```

```
{
  "word": "fuuuuck",
  "normalized": "fuck",
  "canonical": "fuck",
  "language": "english",
  "severity": "high",
  "start": 12.34,
  "end": 12.81,
  "source": "dictionary",
  "confidence": 0.95,
  "variants": ["fuuuuck", "f*ck", "f.u.c.k"]
}
```

Save this as `/schemas/detection.json` in the repo root. If the shape needs to change, it's a team discussion, not a solo decision — everyone's code depends on it.

3. Git Workflow

- `main` — always working / demo-able. Never push broken code directly.
 - One branch per person per feature, e.g. `p1-detection-pipeline`, `p2-celery-export`, `p3-timeline-ui`.
 - PR into `main` at the end of each week. Team lead (P2) reviews/merges — even a quick checklist beats no review:
 - ☐ Does it match the JSON schema (if touching detections)?
 - ☐ Does it run without crashing on the demo test file?
 - ☐ Any new env vars / setup steps documented in README?
 - Keep `README.md` up to date with setup steps and shared conventions — since everyone is prompting AI coding tools independently, a clear README means all three AI assistants get the same context instead of three different guesses.
-

4. Milestones (mapped to Scope §5 Definition of Done)

Week 1 — Foundations

- Lock JSON schema (§2 above) and DB schema.
- P1: Whisper transcribing a test file (English/Hindi/Hinglish sample).
- P2: FastAPI + Postgres + Celery/Redis scaffolding, auth endpoints.
- P3: Static UI with mock detection data (upload screen, dashboard, timeline view).

Week 2-3 — Core Build

- P1: Normalization (§3.1) + dictionary (§3.2) + regex/fuzzy matching (§3.3) working on English, Hindi, and Hinglish test sentences.
- P2: Celery job pipeline wired to FFmpeg/PyDub beep+mute; export endpoints.

- P3: Connect UI to real API (still mocked detection data is fine at this stage).

Week 4 — Integration

- Wire P1's real detection output → P2's pipeline → P3's UI.
- Test against the exact demo file from Scope §5: one English + one Hindi + one Hinglish sentence, each profane, including one stretched spelling (`fuuuuck`) and one abbreviation (`bc`).

Week 5+ — Polish & Buffer

- Bug fixing, sync-accuracy testing (target: <100ms drift), export polish.
- If behind schedule, cut fuzzy matching (§3.4) first — it's explicitly "Should-Have, not Must-Have" per the scope doc.

5. Communication

- Short daily async check-in (5 lines is enough): *did X, blocked on Y, doing Z tomorrow.*
- Flag schema or contract changes immediately — silent drift between AI-generated code and what teammates expect is the main risk for a small, new team moving fast with AI tools.

6. Cut from MVP (don't build these — see original Scope Checklist §1 for full list)

Admin panel, analytics/toxicity scoring, RBAC, OAuth login, guest mode, AI voice cloning, custom sound upload, real-time/live features, subtitle translation, storage/credits UI, and context-aware LLM classification (IndicBERT/XLM-R) are all deferred to "Future Scope." Stay disciplined here — scope creep is called out as a "High Impact / High Likelihood" risk in the Product Vision doc.